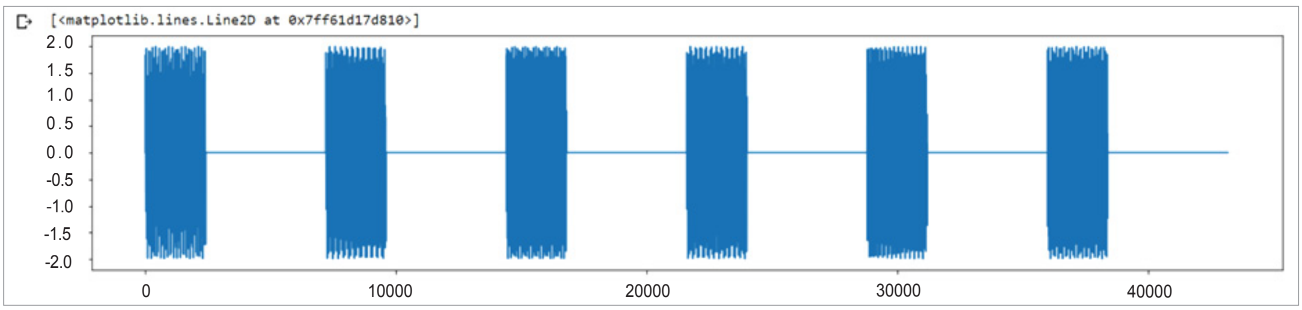

Fig. 1: Audio signals graph

waves, one chosen from one of four low-frequency tones, and one chosen from one of four high-frequency tones, where we are mapping each number to a tuple of its coprime frequencies, as shown in the table. The table shows how digits can be converted to frequency ranges, of which the upper and lower bounds are given.

For example, when you press the button '1' on a phone keypad, a signal consisting of a sine wave at 697Hz plus one at 1209Hz is generated. To fiddle around with DTMF and its encoding and decoding, we have written a library. The library has the functions *dtmf_encoder()* to convert the input number to its corresponding DTMF audio, and *decode_dtmf()* to decode the same.

Okay, now let us try to encode a number like 76589410. We simply write the command:

```
tone_2=dtmf_encoder('76589410')
IPython.display. Audio (tone_2, rate=Fs)
```

In the above command, we are converting each digit into DTMF audio signals by taking sine waves at the required frequencies.

For convenience, plotting the graph of such audio signals taking the sampling rate Fs = 24000Hz is shown in Fig. 1.

It is clearly visible from the plot that each number is a superposition of two different sine waves at two different frequencies.

Now let's encode a number and